

Unit 14: More on Exception Handling

CONTENTS

Objectives

Introduction

14.1 C++ standard exceptions

14.2 Re-throwing Exceptions

14.3 Exceptions in Constructors and Destructors

14.4 Controlling Uncaught Exceptions

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Recognize errors and exception
- Understand Exception Handling
- Analyze C++ standard exceptions
- Describe re-throwing exception

Introduction

We know that it is very rare that a program works correctly first time. It might have bugs. The two most common types of bugs are logic errors and syntactic errors. The logic errors occur due to poor understanding of the problem and solution procedure. The syntactic errors arise due to poor understanding of the language itself. We can detect these errors by using exhaustive debugging and testing procedures.

We often come across some peculiar exceptions problems other than logic or syntax errors. They are known as exceptions. Exceptions are run time anomalies or unusual conditions that a program may encounter while executing. Anomalies might include conditions such as division by zero, access to an array outside of its bounds, or running out of memory or disk space. When a program encounters an exceptional condition, it is important that it is identified and dealt with effectively. ANSI C++ provides built-in language features to detect and handle exceptions which are basically run time errors.

Exception handling was not part of the original C++. It is a new feature added to ANSI C++. Today, almost all compilers support this feature. C++ exception handling provides a type-safe, integrated approach, for coping with the unusual predictable problems that arise while executing a program.



Did you know?

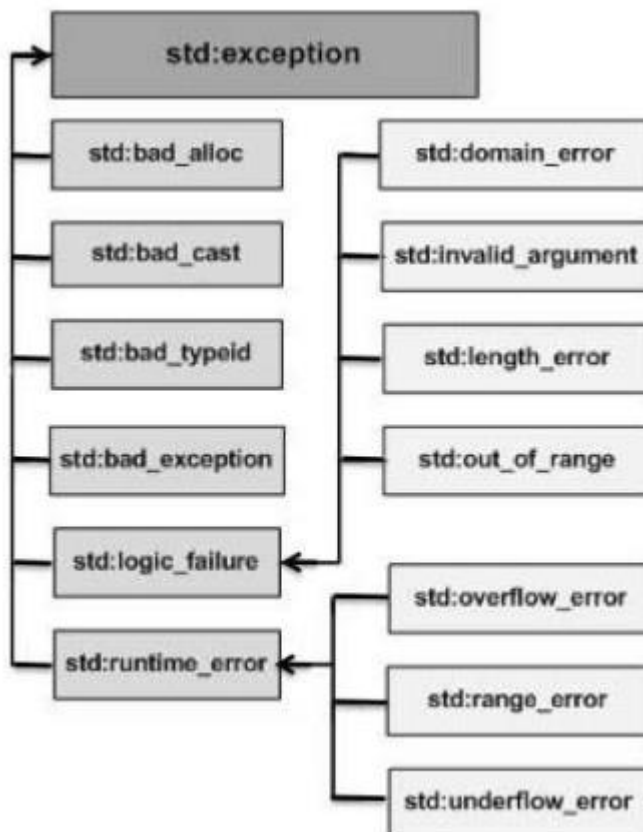
Error: Error is an illegal operation performed by the user which results in abnormal working of the program.

Advantages of Exception Handling

- Exception handling helps programmers to create reliable systems.
- Exception handling separates the exception handling code from the main logic of program.
- Exceptions can be handled outside of the regular code by throwing the exceptions from a function definition or by re-throwing an exception.
- Functions can handle only the exceptions they choose i.e., a function can throw many exceptions, but may choose handle only some of them.

14.1 C++ standard exceptions

C++ provides a list of standard exceptions defined in `<exception>` which we can use in our programs. These are arranged in a parent-child class hierarchy shown below:



Exception	Description
<code>std::exception</code>	An exception and parent class of all the standard C++ exceptions.
<code>std::bad_alloc</code>	This can be thrown by new .
<code>std::bad_cast</code>	This can be thrown by dynamic_cast .
<code>std::bad_exception</code>	This is useful device to handle unexpected exceptions in a C++ program
<code>std::bad_typeid</code>	This can be thrown by typeid .

Unit 14: More on Exception Handling

std::logic_error	An exception that theoretically can be detected by reading the code.
std::domain_error	This is an exception thrown when a mathematically invalid domain is used
std::invalid_argument	This is thrown due to invalid arguments.
std::length_error	This is thrown when a too big std::string is created
std::out_of_range	This can be thrown by the at method from for example a std::vector and std::bitset<>::operator[]().
std::runtime_error	An exception that theoretically can not be detected by reading the code.
std::overflow_error	This is thrown if a mathematical overflow occurs.
std::range_error	This is occurred when you try to store a value which is out of range.
std::underflow_error	This is thrown if a mathematical underflow occurs.

Define New Exceptions:

You can define your own exceptions by inheriting and overriding exception class functionality. Following is the example, which shows how you can use std::exception class to implement your own exception in standard way: -



Lab exercise

```
#include <iostream>
#include <exception>
using namespace std;
struct MyException : public exception
{
    const char * what () const throw ()
    {
        return "C++ Exception";
    }
};

int main()
```

```

{
try
{
throw MyException();
}
catch(MyException& e)
{
std::cout << "MyException caught" << std::endl; std::cout << e.what() << std::endl;
}
catch(std::exception& e)
{
//Other errors
}
}

```

This would produce the following result:

MyException caught

C++ Exception

Here, what() is a public method provided by exception class and it has been overridden by all the child exception classes. This returns the cause of an exception.



Lab exercise

//Program - exception of type bad_typeid

```

#include <iostream>
#include <exception>
#include <typeinfo>
using namespace std;
class A {virtual fun() {} };
int main () {
    try {
        A * a = NULL;
        typeid (*a);
    }
    catch (exception& e)
    {
        cout << "Exception: " << e.what();
    }
    return 0;
}

```

Output

```
Exception: std::bad_typeid
Process returned 0 (0x0)   execution time : 0.037 s
Press any key to continue.
```

14.2 Re-throwing Exceptions

If you want to rethrow an exception from within an exception handler, you can do so by calling `throw` by itself, with no exception. This causes the current exception to be passed on to an outer try/catch sequence. An exception can be rethrown only from within a catch block or from any function called from within that block. When you rethrow an exception, it will not be recaptured by the same catch statement. It will propagate to the immediately enclosing try/catch sequence.

- If a catch block cannot handle the particular exception it has caught, we can rethrow the exception.
- The rethrow expression causes the originally thrown object to be rethrown.



Lab exercise

//Program – rethrowing exception

```
#include <iostream>
using namespace std;
int main()
{
    try
    {
        int a, b;
        cout<<"Enter two integer values: ";
        cin>>a>>b;
        try
        {
            if(b == 0)
            {
                throw b;
            }
            else
            {
                cout<<"Division of a and b is "<<(a/b);
            }
        }
        catch(...)
        {
            throw;
        }
    }
}
```

```

}
catch(int)
{
    cout<<"Second value cannot be zero";
}
return 0;
}

```

Output

Run 1

```

Enter two integer values: 12 0
Second value cannot be zero
Process returned 0 (0x0)   execution time : 4.307 s
Press any key to continue.

```

Run 2

```

Enter two integer values: 12 2
Division of a and b is 6
Process returned 0 (0x0)   execution time : 2.522 s
Press any key to continue.

```

14.3 Exceptions in Constructors and Destructors

It is possible that exceptions might raise in a constructor or destructors. If an exception is raised in a constructor, memory might be allocated to some data members and might not be allocated for others. This might lead to memory leakage problem as the program stops and the memory for data members stays alive in the RAM.

Similarly, when an exception is raised in a destructor, memory might not be deallocated which may again lead to memory leakage problem. So, it is better to provide exception handling within the constructor and destructor to avoid such problems. Following program demonstrates handling exceptions in a constructor and destructor:



Lab exercise

```

//Program
#include<iostream>
using namespace std;
class Divide
{
    private:
        int *x;
        int *y;

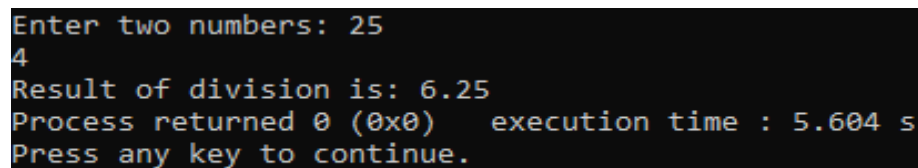
```

```
public:
    Divide()
    {
        x = new int();
        y = new int();
        cout<<"Enter two numbers: ";
        cin>>*x>>*y;
        try
        {
            if(*y == 0)
            {
                throw *x;
            }
        }
        catch(int)
        {
            delete x;
            delete y;
            cout<<"Second number cannot be zero!"<<endl;
            throw;
        }
    }
    ~Divide()
    {
        try
        {
            delete x;
            delete y;
        }
        catch(...)
        {
            cout<<"Error while deallocating memory"<<endl;
        }
    }
    float division()
    {
        return (float)*x / *y;
    }
};

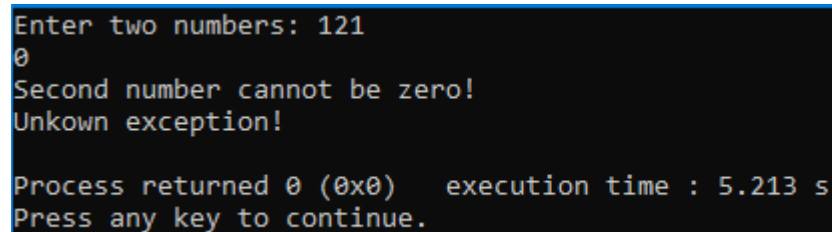
int main()
{
```

```
    try
    {
        Divide d;
        float res = d.division();
        cout<<"Result of division is: "<<res;

    }
    catch(...)
    {
        cout<<"Unkown exception!"<<endl;
    }
    return 0;
}
```

Output**Run 1**A screenshot of a terminal window showing the output of a C++ program. The text is as follows:

```
Enter two numbers: 25
4
Result of division is: 6.25
Process returned 0 (0x0)   execution time : 5.604 s
Press any key to continue.
```

Run 2A screenshot of a terminal window showing the output of a C++ program. The text is as follows:

```
Enter two numbers: 121
0
Second number cannot be zero!
Unkown exception!

Process returned 0 (0x0)   execution time : 5.213 s
Press any key to continue.
```

14.4 Controlling Uncaught Exceptions

- Whenever an exception arises in C++, it is handled as per the behaviour defined using the try-catch block.
- However, there is often the case when an exception is thrown but isn't caught because the exception handling subsystem fails to find a matching catch block for that particular exception.

In Such cases, following actions take place

- The exception handling subsystem calls the function: `unexpected°`. This function, provided by the default C++ library, defines the behavior when an uncaught exception arises. By default, `unexpected` calls `terminate°`.
- The `terminate` function defines the actions that are to be performed during process termination. This, by default, calls `abort0`.

- The process is aborted.



Lab exercise

```
//      Program
#include <iostream>
#include <exception>
#include <cstdlib>
using namespace std;
void myterminate() {
    cerr << "terminate handler called\n";
    abort();
}
int main (void) {
    set_terminate(myterminate);
    throw 0;
    return 0;
}
```

Output

```
terminate handler called

This application has requested the Runtime to terminate it in an unusual way.
Please contact the application's support team for more information.

Process returned 3 (0x3)   execution time : 1.198 s
Press any key to continue.
```

Summary

- A try block may throw an exception directly or invoke a function that throws an exception. Irrespective of location of the throw point, the catch block is placed immediately after the try block.
- We can place two or more catch blocks together to catch and handle multiple types of exceptions thrown by a try block.
- It is also possible to make a catch statement to catch all types of exceptions using ellipses as its argument.
- We may also restrict a function to throw only a set of specified exceptions by adding a throw specification clause to the function definition.

Keywords

Error - Error is an illegal operation performed by the user which results in abnormal working of the program.

Object Oriented Programming Using C++

Exception - An exception is a problem that arises during the execution of a program. A C++ exception is a response to an exceptional circumstance that arises while a program is running, such as an attempt to divide by zero.

Run time error - They are also known as exceptions. An exception caught during run time creates serious issues.

Throw- when a program encounters a problem, it throws an exception. The throw keyword helps the program perform the throw.

Catch- a program uses an exception handler to catch an exception. It is added to the section of a program where you need to handle the problem. It's done using the catch keyword.

Try- the try block identifies the code block for which certain exceptions will be activated. It should be followed by one/more catch blocks.

Self Assessment

1. If inner catch handler is not able to handle the exception then_____.
 - A. Compiler will look for outer try handler
 - B. Program terminates abnormally
 - C. Compiler will check for appropriate catch handler of outer try block
 - D. None of the above
2. What is Re-throwing an exception means in C++?
 - A. An exception that is thrown again as it is not handled by that catching block
 - B. An exception that is caught twice
 - C. An exception that is not handled in one caught hence thrown again
 - D. All of the mentioned
3. Which header file is used to declare the standard exception?
 - A. #include<exception>
 - B. #include<except>
 - C. #include<error>
 - D. #include<exce>
4. Where are standard exception classes grouped?
 - A. namespace std
 - B. error
 - C. catch
 - D. final
5. Which of the following is not a standard exception?
 - A. bad_cast
 - B. bad_exception
 - C. #define
 - D. bad_function_call
6. Which of the following is a standard exception?
 - A. #include<iostream>
 - B. bad_cast
 - C. All of above

- D. None of above
7. If a catch block cannot handle the particular exception it has caught, we can rethrow the exception.
- A. True
- B. False
8. How to handle error in the destructor?
- A. Throwing
- B. Terminate
- C. Both throwing & terminate
- D. None of the above
9. Terminate function will called when we have a uncaught exception?
- A. True
- B. False
10. How to handle exception in constructor, in c++?
- A. We have to return an exception
- B. We have to throw an exception
- C. All of above
- D. None of the above
11. What is the role of a constructor in classes?
- A. To modify the data whenever required
- B. To destroy an object
- C. To initialize the data members of an object when it is created
- D. To call private functions from the outer world
12. Destructor has the same name as the constructor and it is preceded by _____. .
- A. !
- B. ?
- C. ~
- D. \$
13. What will not be called when the terminate() is arised in constructor?
- A. main()
- B. class
- C. destructor
- D. none of the mentioned
14. What function will be called when we have a uncaught exception?
- A. Catch
- B. throw
- C. terminate
- D. none of the mentioned
15. Catch function will called when we have a uncaught exception?
- A. True
- B. False

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. C | 2. D | 3. A | 4. A | 5. C |
| 6. B | 7. A | 8. B | 9. A | 10. B |
| 11. C | 12. D | 13. C | 14. C | 15. B |

Review Questions

1. What do you mean by an uncaught exception?
2. What is an exception? How it is different from error.
3. What is an exception?
4. Explain standard exceptions in C++ with suitable example.
5. List the advantages of exception handling.
6. What are the basic implementations handling exceptions using constructor and destructor?
7. Explain in detail “controlling uncaught exception”.
8. Write a program using C++ that demonstrate working of exception handling.

**Further Readings**

E Balagurusamy; Object-oriented Programming with C++; Tata Mc Graw-Hill.

Herbert Schildt; The Complete Reference C++; Tata Mc Graw Hill.

Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.

**Web Links**

<https://study.com/academy/lesson/practical-application-for-c-plus-plus-programming->

<https://www.codecademy.com/learn/learn-c-plus-plus>

<https://www.geeksforgeeks.org/exception-handling-c/>

LOVELY PROFESSIONAL UNIVERSITY

Jalandhar-Delhi G.T. Road (NH-1)

Phagwara, Punjab (India)-144411

For Enquiry: +91-1824-521360

Fax.: +91-1824-506111

Email: odl@lpu.co.in